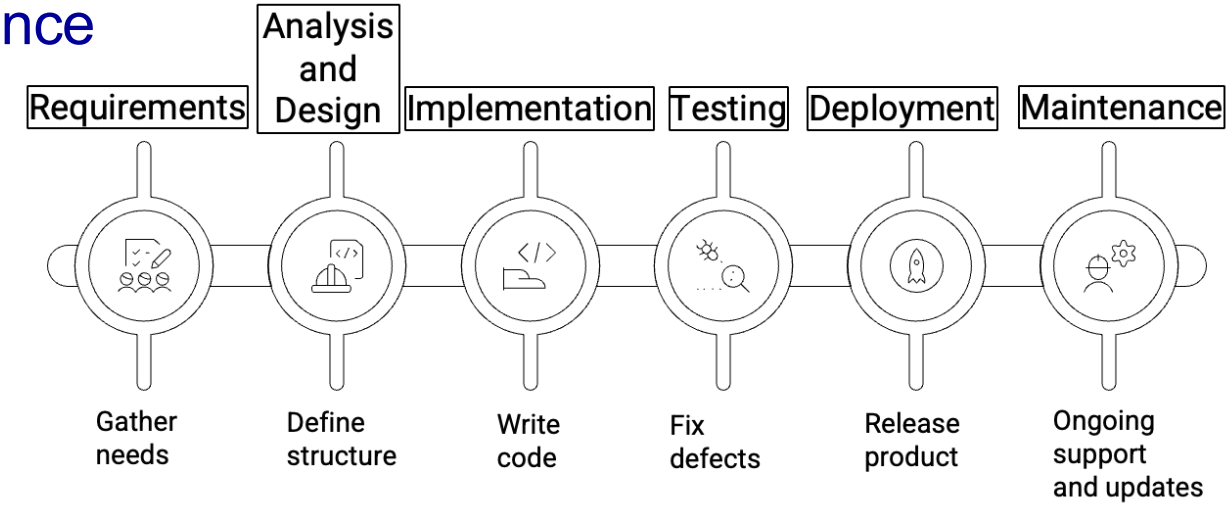
Lecture Session

Software Processes

- Software Development Activities
- Software Development Model
- Waterfall vs Agile

Software process

- A software process is a set of structured activities that leads to the production of the software.
- Many different software processes but all involve:
  - Requirements
  - Development: Analysis, Design, Implementation
  - Validation: Testing and Deployment
  - Maintenance



Outcomes of each activity

- **Requirements:** requirements specification
- **Analysis:** a set of analysis models
- **Design:** detailed design documentation
- **Implementation:** executable version of the software
- **Testing:** fully tested software
- **Deployment:** working software in real environment

Maintenance(Evolution)

- Aim: keeping the system operational after delivery to the customer; changing the system in response to changing customer needs.
  - Corrective: identification and removal of faults
  - Adaptive: modifications needed to achieve interoperability with other systems and platforms
  - Perfective: incorporation of new features, improved performance and other modifications
  - Preventive: modifications to mitigate possible degradation of usability, maintainability, etc

Software process models

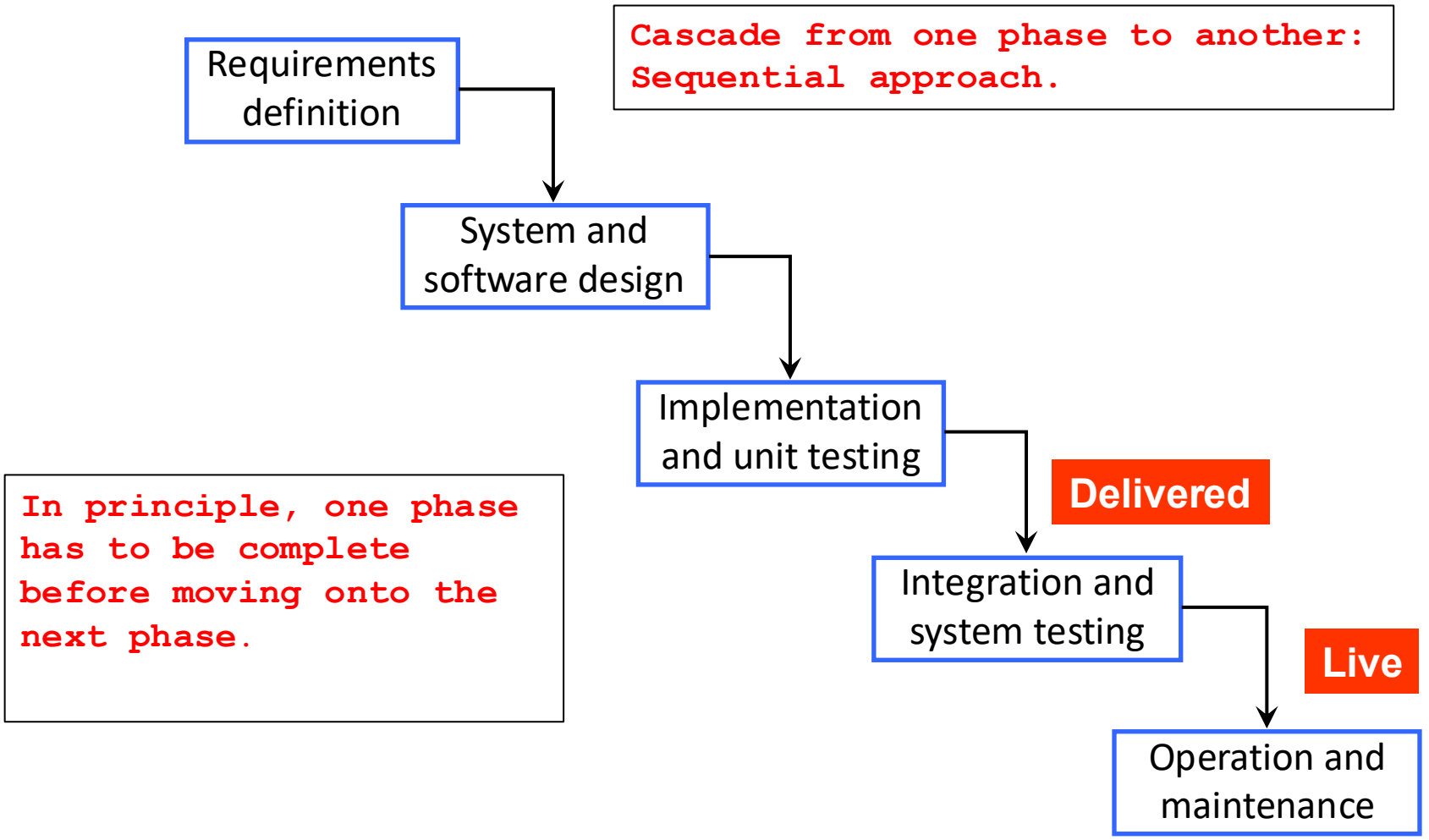
- Depends on the system, the activities can be:
  - organised in sequence
  - organised as interleaved
  - organised concurrently
- Must be modelled in order to be managed.
- Software Process model
  - A simplified representation of a software process – an abstract representation.

Classic Models

- Waterfall model (traditional)
  - Separate and distinct phases
- Agile model (modern)
  - Iterations

There are many other software process models, e.g. Evolutionary model, Rational Unified Model, Spiral Model, V Model, Formal Model…

Waterfall model and its phases



Waterfall model benefits

- Easy to monitor the progress
  - After a small number of iterations, freeze parts of the development and continue with later phases.
- Documentation is well produced at each stage.
- Structured approach.
- Specialised teams can be used at each stage of the lifecycle.

Waterfall model problems

- Inflexible
  - Difficulty of accommodating change after the process is underway.
- Time consuming
  - Real projects rarely follow the sequential flow.
  - A working version of the system will not be available until late in the project time-span.
- Minimises impact of global understanding over the lifecycle of a project.
- Not realistic.

Waterfall model – suitable projects

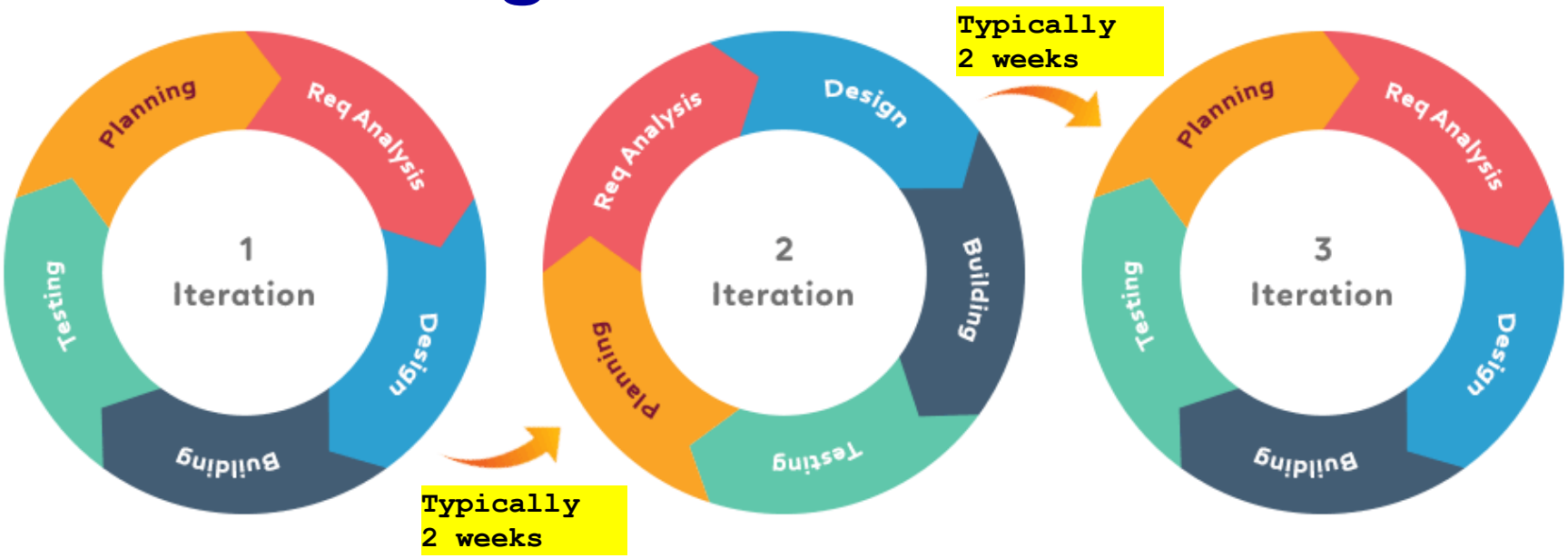
- This model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
  - Few business systems have stable requirements!
- Adaptation or enhancement of existing system.
- In high risk, safety critical systems e.g. air traffic control, quality is key.

Examples:
aircraft systems, space systems, nuclear power systems, critical health systems, power and telecommunications systems…

The Agile Process

- The processes of specification, design, implementation and testing are concurrent, referred to as an iteration:
  - no detailed specification;
  - design documentation is minimised.
- The system is developed in a series of increments
  - End users evaluate each increment and make proposals for later increments.
- End users are involved
  - System user interfaces are usually developed using an interactive development system.

Agile iterations



The release cycle at each iteration:

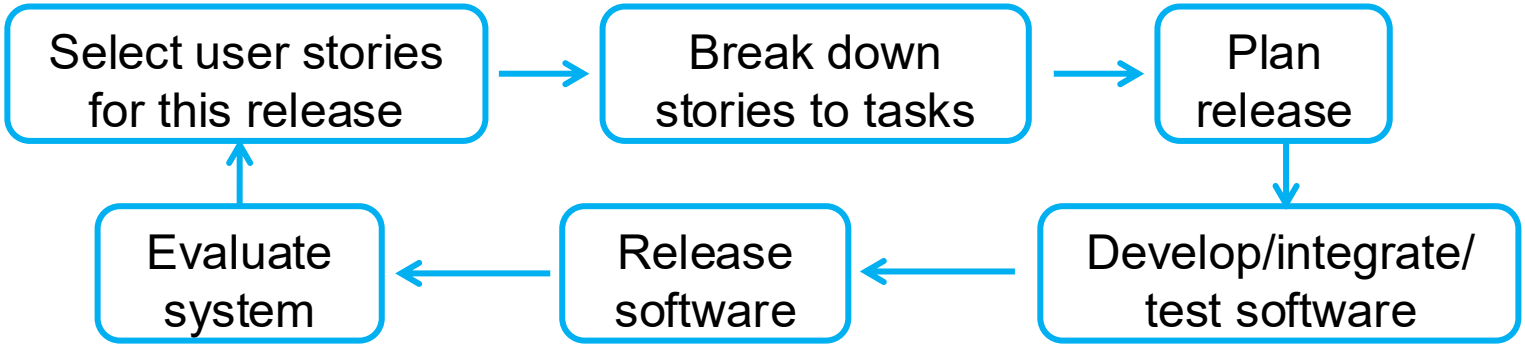
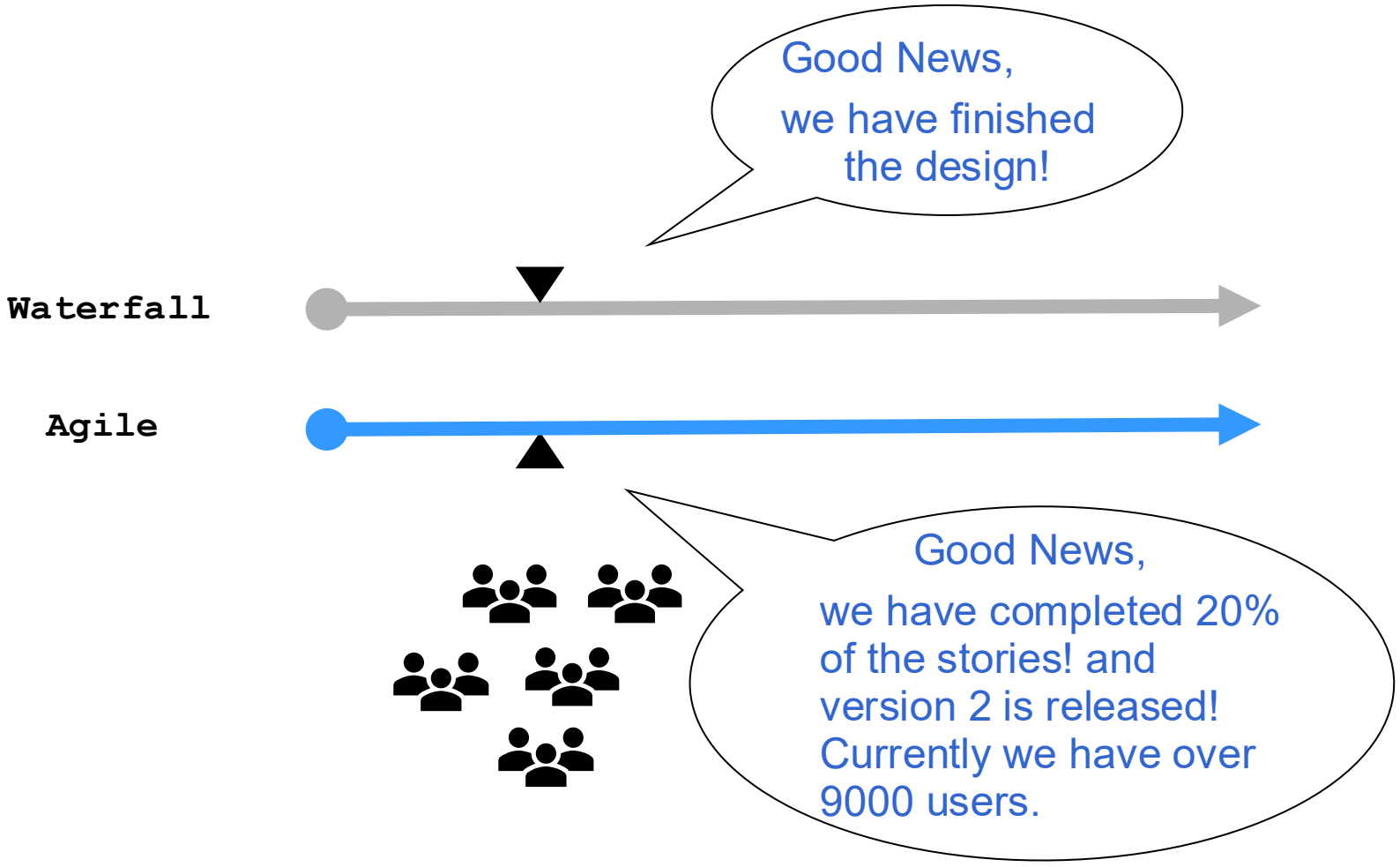


Image cc medium.com/

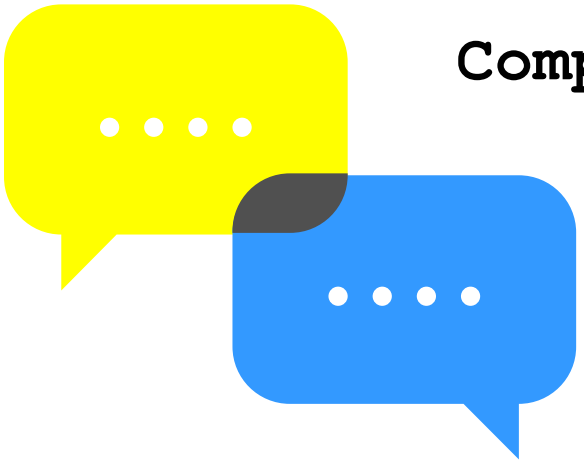
Agile – suitable projects

- Small or medium-sized product.
- Requirement is not clear and/or keep changing.
- Rapid delivery.
- A clear commitment from the customer to become involved in the development process
- Not a lot of external rules and regulations that affect the software

Waterfall vs Agile



Practical Exercises Session (15-20 minutes in class)



Compare Waterfall and Agile

Lecture Session

Agile Software Development

- What is Agile
- Agile Principles

What is Agile?

- Agile is a set of best practices in software development based on *Scrum*, *Extreme Programming,* Crystal Clear, DSDM, *Lean* and others.
- The set includes:
  - Iteration, Test Driven Development, continuous integration, refactoring, pair programming, story card/wall, automation test, feedback, stand up, retrospective and showcase.

Focus of Agile

- The focus of Agile:
  - Focus on the code rather than the analysis/design.
  - Are based on an iterative approach to software development.
  - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.

Lightweight, highly iterative, short iterations, test-driven, focus on value to customer, frequent releases, ability to cope with change

Integration and release

- Frequent integration:
  - multiple builds per day;
  - everyone involved;
  - automated tool support.
- Small & Frequent Releases:
  - Team releases running, tested software delivering business value, as determined by the customer, at every iteration.

The Agile Manifesto

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

Agile team

- Agile focuses on developers (programmers) but need other roles, e.g. business analyst, project manager, tester, …
- Small, co-located, multi-disciplinary team, members are usually working around a table
  - Easy communication
- Collective code ownership
- Common vision of system
- Common coding standard

Dos and Do NOTs

- Agile needs necessary documentation
  - BUT need to ensure that every document has an audience.
- Agile encourages good practices
  - BUT need to ensure that every practice solves a problem.
- Drop anything without value.
- Don’t over design the system.

Principles of Agile (Brief)

- Customer involvement
  - Closely involved throughout the development
- Incremental delivery
  - Customer specifies each increment
- People, not process
  - Skills, the own way
- Embrace change
  - Expect change
- Maintain simplicity
  - Both the system and the process

Principles of Agile (*more detail*)

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

https://www.agilealliance.org/agile101/12-principles-behind-the-agile-manifesto/

Agile Problems (1/2)

- Problems
  - It can be difficult to keep the interest of customers who are involved in the process.
  - Team members may be unsuited to the intense involvement that characterises agile methods.

Agile Problems (2/2)

- Problems
  - Prioritising changes can be difficult where there are multiple stakeholders.
  - Maintaining simplicity requires extra work.
  - Contracts may be a problem as with other approaches to iterative development.

Agile requires…

- Experience
  - Has experienced leadership
  - Hire an experienced team
- Working environment
  - Motivated stable teams
  - Supportive management
  - Team dynamics is critical
- Value thinking
- Effective and Efficient communication
- Information sharing
- Tools and Automation

Practical Exercises Session (15-20 minutes in class)

Understanding Agile

Summary

- Software Processes
  - Software Development Activities
  - Software Development Model
  - Waterfall vs Agile
- Agile Software Development
  - What is Agile
  - Agile Principles